

LAPORAN PRAKTIKUM STRUKTUR DATA

Binary Tree dan Graph



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 10 Juni 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum **“Binary Tree dan Graph”** ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi dan pemahaman hasil kegiatan praktikum yang bertujuan untuk memahami bagaimana konsep dari Binary Tree dan Graph. Penulis juga terbuka untuk menerima saran dan kritik guna perbaikan di masa mendatang.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah struktur data, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.

Padang, 10 Juni 2026

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM STRUKTUR DATA	i
KATA PENGANTAR.....	ii
DAFTAR PUSTAKA.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Landasan teori	3
2.2 Langkah pengerjaan.....	4
BAB III KESIMPULAN.....	21
DAFTAR PUSTAKA.....	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digital saat ini, pengelolaan dan pengolahan data menjadi fondasi utama dalam pengembangan sistem komputasi modern. Struktur data memegang peranan yang sangat krusial dalam mengorganisasi, mengelola dan menyimpan data agar dapat diakses serta dimodifikasi secara efisien. Diantara berbagai jenis struktur data, struktur data non-linear seperti *Binary Tree* (Pohon Biner) dan *Graph* (Graf) memiliki signifikansi yang sangat tinggi. Kedua struktur ini mampu merepresentasikan hubungan hierarkis dan jaringan yang kompleks, yang tidak dapat ditangani secara optimal oleh struktur data linear seperti array atau linked list

Oleh karena itu, laporan praktikum ini disusun sebagai dokumentasi sistematis dari proses implementasi, pengujian, dan analisis konsep *Binary Tree* dan *Graph*. Kegiatan praktikum ini bertujuan untuk memperkuat kompetensi praktikan dalam menerjemahkan teori struktur data ke dalam solusi komputasi yang nyata dan efisien, serta memberikan pemahaman praktis mengenai langkah-langkah penulisan dan manipulasi struktur data non-linear dalam pemrograman.

1.2 Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Memahami struktur Binary Tree dan Graph
2. Memahami langkah-langkah menulis Binary Tree dan Graph

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Meningkatkan pemahaman tentang Binary Tree dan Graph Meningkatkan pemahaman bagaimana cara menulis Binary Tree dan Graph

BAB II

PEMBAHASAN

2.1 Landasan teori

Struktur data merupakan metode pengorganisasian dan penyimpanan data di dalam komputer agar data tersebut dapat diakses dan dimodifikasi secara efisien. Sementara struktur data linear seperti array dan linked list cocok untuk data yang berurutan, struktur data non-linear seperti Tree (Pohon) dan Graph (Graf) dirancang untuk merepresentasikan hubungan hierarkis dan jaringan yang lebih kompleks. Pemahaman mendalam terhadap struktur data non-linear ini sangat penting dalam ilmu komputer karena menjadi fondasi bagi berbagai algoritma canggih, mulai dari pengindeksan basis data hingga penentuan rute terpendek dalam sistem navigasi.

Binary Tree (Pohon Biner) adalah salah satu bentuk struktur data tree di mana setiap simpul (node) memiliki paling banyak dua anak, yang secara konvensional disebut sebagai anak kiri (left child) dan anak kanan (right child). Simpul paling atas dalam hierarki disebut sebagai akar (root), sedangkan simpul yang tidak memiliki anak disebut sebagai daun (leaf). Salah satu varian Binary Tree yang paling banyak digunakan adalah Binary Search Tree (BST), di mana nilai pada setiap simpul anak kiri selalu lebih kecil daripada nilai simpul induknya, dan nilai pada anak kanan selalu lebih besar. Properti ini memungkinkan operasi pencarian, penyisipan, dan penghapusan data dilakukan dengan efisiensi waktu rata-rata $O(\log n)$, menjadikannya sangat unggul untuk pengelolaan data dinamis. Untuk menelusuri atau memproses seluruh elemen dalam Binary Tree, digunakan metode penelusuran (traversal) yang terdiri dari Pre-order (mengunjungi akar terlebih dahulu), In-order (mengunjungi anak kiri, lalu akar, lalu anak kanan), dan Post-order (mengunjungi anak-anak terlebih dahulu sebelum akar).

Di sisi lain, Graph adalah struktur data yang terdiri dari kumpulan hingga simpul (vertices atau nodes) dan sisi (edges) yang menghubungkan pasangan simpul tersebut. Secara matematis, Graph didefinisikan sebagai $G = (V, E)$, di mana V adalah himpunan simpul dan E adalah himpunan sisi. Graph dapat diklasifikasikan menjadi beberapa jenis, seperti Graph berarah (directed graph) di mana sisi memiliki orientasi, Graph tak berarah (undirected graph), serta Graph berbobot (weighted graph) di mana setiap sisi memiliki nilai atau biaya tertentu. Struktur ini sangat ideal untuk memodelkan masalah dunia

nyata, seperti jaringan sosial, topologi jaringan komputer, peta jalan, dan dependensi antar tugas.

Implementasi Graph dalam pemrograman umumnya dilakukan menggunakan dua representasi utama, yaitu Adjacency Matrix (Matriks Ketetanggaan) dan Adjacency List (Daftar Ketetanggaan). Adjacency Matrix menggunakan array dua dimensi yang efisien untuk Graph padat (dense graph), sedangkan Adjacency List menggunakan kumpulan linked list yang lebih hemat memori untuk Graph renggang (sparse graph). Untuk menelusuri simpul-simpul dalam Graph, dua algoritma fundamental yang paling sering digunakan adalah Breadth-First Search (BFS) yang menelusuri Graph level demi level menggunakan struktur data antrean (queue), dan Depth-First Search (DFS) yang menelusuri sedalam mungkin pada setiap cabang sebelum melakukan backtracking dengan memanfaatkan struktur data tumpukan (stack) atau rekursi. Penguasaan terhadap konsep dan implementasi Binary Tree serta Graph ini menjadi kompetensi inti yang harus dikuasai dalam mata kuliah Struktur Data.

2.2 Langkah pengerjaan

- Node_2511532017
 1. Buka eclipse dan buat package baru dengan nama pekan9_2511532017 dan class Node_2511532017.
 2. Setelah dibuat, kita akan membuat Node.

```
1 package pekan9_2511532017;  
2  
3 public class Node_2511532017 {  
4     int data_2017;  
5     Node_2511532017 left_2017;  
6     Node_2511532017 right_2017;
```

3. Atribut data_2017 bertipe int menyimpan nilai atau data aktual, left_2017 variabel bertipe Node_2511532017 yang berfungsi sebagai penunjuk ke simpul anak di sebelah kiri dan right_2017 variabel bertipe Node_2511532017 yang berfungsi sebagai penunjuk ke simpul anak di sebelah kanan.

```

7 public Node_2511532017 (int data_2017) {
8     this.data_2017 = data_2017;
9     left_2017 = null;
10    right_2017 = null;
11 }

```

4. Konstruktor dimana akan dipanggil saat objek node baru dibuat, `this.data_2017 = data_2017`; mengisi nilai data simpul dengan nilai yang diberikan saat membuat objek, `left_2017 = null`; dan `right_2017 = null`; karena belum memiliki anak maka keduanya null

```

12 public void setLeft_2511532017 (Node_2511532017 node_2017) {
13     if (left_2017 == null)
14         left_2017 = node_2017;
15 }
16 public void setRight_2511532017 (Node_2511532017 node_2017) {
17     if (right_2017 == null)
18         right_2017 = node_2017;
19 }
20 public Node_2511532017 getLeft_2017 () {
21     return left_2017;
22 }
23 public Node_2511532017 getRight_2017 () {
24     return right_2017;
25 }
26 public int getData_2017 () {
27     return data_2017;
28 }
29 public void setData_2017 (int data_2017) {
30     this.data_2017 = data_2017;
31 }

```

5. Setter untuk menambahkan anak kiri atau anak kanan, getter untuk mengambil nilai

```

void printPreorder_2017 (Node_2511532017 node_2017) {
    if (node_2017 == null)
        return;
    System.out.print(node_2017.data_2017 + " ");
    printPreorder_2017 (node_2017.left_2017);
    printPreorder_2017 (node_2017.right_2017);
}

```

6. Method penelusuran Preorder, mencetak nilai simpul saat ini (root) dahulu lalu menelusuri seluruh cabang kiri baru kemudian cabang kanan

```

void printPostorder_2017 (Node_2511532017 node_2017) {
    if (node_2017 == null)
        return;
    printPostorder_2017 (node_2017.left_2017);
    printPostorder_2017 (node_2017.right_2017);
    System.out.print(node_2017.data_2017 + " ");
}

```


7. Method postorder, menelusuri cabang kanan sampai habis dan terakhir mencetak nilai simpul saat ini (root)

```

5 void printInorder_2017 (Node_2511532017 node) {
6     if (node == null)
7         return;
8     printInorder_2017 (node.left_2017);
9     System.out.print(node.data_2017 + " ");
10    printInorder_2017 (node.right_2017);
11
12 }

```

8. Method Inorder, menelusuri cabang kiri laly mencetak simpul saat ini, baru mencetak cabaang kanan.

```

public String print_2511532017 () {
    return this.print_2017("", true, "");
}

```

9. Method ini dipanggil dari luar (misalnya dari main) untuk memulai proses visualisasi. Ia memanggil metode rekursif print_2017 dengan nilai awal: prefix kosong, isTail bernilai true (menandakan ini adalah root/akar), dan sb (string builder) kosong

```

public String print_2017 (String prefix_2017, boolean isTail_2017, String sb_2017) {
    if (right_2017 != null) {
        right_2017.print_2017(prefix_2017 + (isTail_2017 ? "| " : " "), false, sb_2017);
    }
    System.out.println(prefix_2017 + (isTail_2017 ? "\\--" : "/--") + data_2017);
    if (left_2017 != null) {
        left_2017.print_2017(prefix_2017 + (isTail_2017 ? " " : "| "), true, sb_2017);
    }
    return sb_2017;
}
}

```

10. Method print_2017 digunakan untuk mencetak pohon

- BTree_2511532017

1. Kita akan membuat class baru untuk BTree_2511532017

```

1 package pekan9_2511532017;
2
3 public class BTree_2511532017 {
4     private Node_2511532017 root_2017;
5     private Node_2511532017 currentNode_2017;

```

2. Atribut root_2017 merupakan variabel penunjuk ke simpul paling atas dari pohon, currentNode_2017 merupakan variabel penunjuk ke simpul ayng sedang aktif

```

6 public BTree_2511532017 () {
7     root_2017 = null;
8 }

```

3. Kontruktor yang akan dipanggil saat objek Btree baru dibuat

```

public boolean search_2017 (int data_2017) {
    return search_2017 (root_2017, data_2017);
}

```

4. public search_2017 berfungsi sebagai wrapper

```

private boolean search_2017 (Node_2511532017 node_2017, int data_2017) {
    if (node_2017.getData_2017() == data_2017)
        return true;
    if (node_2017.getLeft_2017() != null)
        if (search_2017(node_2017.getLeft_2017(), data_2017))
            return true;
    if (node_2017.getRight_2017() != null)
        if (search_2017(node_2017.getRight_2017(), data_2017))
            return true;
    return false;
}

```

5. private search_2017 merupakan implementasi dari DFS

```

public void printInorder_2017() {
    root_2017.printInorder_2017(root_2017);
}
public void printPreorder_2017() {
    root_2017.printPreorder_2017(root_2017);
}
public void printPostorder_2017() {
    root_2017.printPostorder_2017(root_2017);
}
public Node_2511532017 getRoot_2017() {
    return root_2017;
}

```

6. method penelusuran yang bertindak sebagai penerus

```

public boolean isEmpty_2017 () {
    return root_2017 == null;
}

```

7. mengecek apakah pohon kosong

```

    public int countNodes_2017() {
        return countNodes_2017 (root_2017);
    }

    private int countNodes_2017 (Node_2511532017 node_2017) {
        int count_2017 = 1;
        if ( node_2017 == null) {
            return 0;
        }else {
            count_2017 += countNodes_2017 (node_2017.getLeft_2017());
            count_2017 += countNodes_2017 (node_2017.getRight_2017());
            return count_2017;
        }
    }
}

```

8. countNodes_2017 method untuk menghitung node dari root

```

public void print () {
    root_2017.print_2511532017();
}

```

9. method untuk visualisasi pohon

```

    public Node_2511532017 getCurrent_2017() {
        return currentNode_2017;
    }

    public void setCurrent (Node_2511532017 node_2017) {
        this.currentNode_2017 = node_2017;
    }

    public void setRoot_2017 (Node_2511532017 root_2017) {
        this.root_2017 = root_2017;
    }

}

```

10. getter dan setter untuk root dan current

- BTreeDriver_2511532017
 1. kita akan membuat class baru untuk BTreeDriver_2511532017

```

1 package pekan9_2511532017;
2
3 public class BTreeDriver_2511532017 {
4
5     public static void main(String[] args) {
6         //membuat Pohon
7         BTree_2511532017 tree_2017 = new BTree_2511532017();
8         System.out.print("Jumlah Simpul awal pohon: ");
9         System.out.println(tree_2017.countNodes_2017());

```

2. BTree_2511532017 tree_2017 = new BTree_2511532017();
membuat objek pohon biner baru dan masih kosong,
countNodes_2017() memanggil metode untuk menghitung
jumlah simpul

```
//menambahkan simpul data 1
Node_2511532017 root_2017 = new Node_2511532017(1);
//meniadikan simpul 1 sebagai root
tree_2017.setRoot_2017(root_2017);
System.out.println("Jumlah simpul jika hanya ada root");
System.out.println(tree_2017.countNodes_2017());
```

3. new Node_2511532017(1) membuat simpul pertama dengan nilai
1, setRoot_2511532017(root_2017) menetapkan simpul 1 sebagai
akar dari tree_2017

```
Node_2511532017 node2 =new Node_2511532017(2);
Node_2511532017 node3 =new Node_2511532017(3);
Node_2511532017 node4 =new Node_2511532017(4);
Node_2511532017 node5 =new Node_2511532017(5);
Node_2511532017 node6 =new Node_2511532017(6);
Node_2511532017 node7 =new Node_2511532017(7);
Node_2511532017 node8 =new Node_2511532017(8);
Node_2511532017 node9 =new Node_2511532017(9);
```

4. membuat objek simpul

```
root_2017.setLeft_2511532017(node2);
node2.setLeft_2511532017(node4);
node2.setRight_2511532017(node5);
node4.setRight_2511532017(node8);
root_2017.setRight_2511532017(node3);
node3.setLeft_2511532017(node6);
node3.setRight_2511532017(node7);
node6.setLeft_2511532017(node9);
```

5. penghubung simpul menggunakan left dan right

```
//set root
tree_2017.setCurrent(tree_2017.getRoot_2017());
System.out.println("menampilkan simpul terakhir: ");
System.out.println(tree_2017.getCurrent_2017().getData_2017());
System.out.println("jumlah simpul; setelah simpul 7 ditambahkan ");
System.out.println(tree_2017.countNodes_2017());
```

6. setCurrent menetapkan currentNode ke posisi root,
getCurrent_2017().getData_2017() mengambil dan mencetak data
dari simpul yang sedang difokuskan, countNodes_2017()
menghitung total seluruh simpul

```

System.out.println("InOrder: ");
tree_2017.printInorder_2017();
System.out.println("\nPreOrder: ");
tree_2017.printPreorder_2017();
System.out.println("\nPostOrder: ");
tree_2017.printPostorder_2017();

```

7. penelusuran pohon

```

System.out.println("\nmenampilkan simpul dalam bentuk pohon");
tree_2017.print();
}
}

```

8. visualisasi pohon

9. output

```

Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
menampilkan simpul dalam bentuk pohon
|
|--7
|
|--3
|   |--6
|   |--9
|--1
|   |--5
|   |--2
|       |--8
|       |--4

```

• GraphTraversal_2511532017

1. Buat class dengan nama GraphTraversal_2511532017

```

1 package pekan9_2511532017;
2 import java.util.*;
3 public class GraphTraversal_2511532017 {
4     private Map<String, List<String>> graph_2017 = new HashMap<>();
5

```

2. Map<String, List<String>> menggunakan HashMap dimana String Merepresentasikan nama sebuah simpul (node), misalnya "A", "B", List<String> Merepresentasikan daftar simpul-simpul tetangga yang terhubung langsung dengan node tersebut.

```

6 //menambahkan edge (graf tak berarah)
7 public void adEdge_2017 (String node1_2017, String node2_2017) {
8     graph_2017.putIfAbsent(node1_2017, new ArrayList<>());
9     graph_2017.putIfAbsent(node2_2017, new ArrayList<>());
10    graph_2017.get (node1_2017).add(node2_2017);
11    graph_2017.get (node2_2017).add(node1_2017);
12 }

```

3. putIfAbsent memeriksa apakah node1 dan node2 sudah ada sebagai Key di dalam HashMap, jika belum buat Value berupa ArrayList kosong untuk node tersebut. Ini mencegah error NullPointerException, get dan add menambahkan node2 ke dalam daftar tetangga milik node1 , menambahkan node1 ke dalam daftar tetangga milik node2

```

//menampilkan graf awal
public void printGraph_2017 () {
    System.out.println("Graf Awal (Adjacency List): ");
    for (String node_2017 : graph_2017.keySet()) {
        System.out.print(node_2017 + " -->");
        List<String> neighbors = graph_2017.get(node_2017);
        System.out.println(String.join(", ", neighbors));
    }
    System.out.println();
}

```

4. method public void printGraph_2017 () digunakan untuk pencetakan graf

```

//DFS rekursif
public void dfs_2017 (String start) {
    Set<String> visited = new HashSet<>();
    System.out.println("Penelusuran DFS: ");
    dfsHelper_2017 (start, visited);
    System.out.println();
}

private void dfsHelper_2017 (String current, Set<String> visited) {
    if (visited.contains(current)) return;
    visited.add(current);
    System.out.print(current + " ");
    for (String neighbor_2017 : graph_2017.getOrDefault(current, new ArrayList<>())) {
        dfsHelper_2017 (neighbor_2017, visited);
    }
}

```

5. method DFS Set<String> visited menggunakan HashSet untuk menyimpan nama-nama node yang sudah dikunjungi, memanggil dfsHelper_2017 sebagai method pembantu

```
//BFS iteratif
public void bfs_2017 (String start_2017) {
    Set <String> visited_2017 = new HashSet <>();
    Queue <String> queue_2017 = new LinkedList <>();
    queue_2017.add(start_2017);
    visited_2017.add(start_2017);
    System.out.println("Penelusuran BFS: ");
    while (!queue_2017.isEmpty()) {
        String current = queue_2017.poll();
        System.out.print(current + " ");
        for (String neighbor : graph_2017.getOrDefault(current, new ArrayList<>())) {
            if (!visited_2017.contains(neighbor)) {
                queue_2017.add(neighbor);
                visited_2017.add(neighbor);
            }
        }
    }
    System.out.println();
}
```

6. method BFS while (!queue_2017.isEmpty()) selama antrian masih berisi node yang akan diproses, keluarkan node paling depan dari antrian simpan current dan cetak

```
}
public static void main (String[] args) {
    GraphTraversal_2511532017 graph_2017 = new GraphTraversal_2511532017 ();

    //contoh graf: A-B, A-C, B-D, B-E
    graph_2017.addEdge_2017("A", "B");
    graph_2017.addEdge_2017("A", "C");
    graph_2017.addEdge_2017("B", "D");
    graph_2017.addEdge_2017("B", "E");

    //cetak graf awal
    System.out.println("Graf awal adalah: ");
    graph_2017.printGraph_2017();

    //lakukan penelusuran
    graph_2017.dfs_2017("A");
    graph_2017.bfs_2017("A");
}
}
```

7. method main
8. output

```
Graf awal adalah:
Graf Awal (Adjacency List):
A -->B,C
B -->A,D,E
C -->A
D -->B
E -->B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

- PetaTempatWisata_2511532017
 1. Kita buat kelas untuk PetaTempatWisata_2511532017

```

1 package pekan9_2511532017;
2
3 import java.awt.*;
4 import java.awt.event.*;
5 import java.util.*;
6 import java.util.List;
7 import javax.swing.*;
8 import javax.swing.border.*;
9
10 public class PetaTempatWisata_2511532017 extends JFrame {
11
12     private static final long serialVersionUID = 1L;
13
14
15     static class GraphTraversal_2017 {
16
17         private Map<String, List<String>> graph_2017 = new LinkedHashMap<>();
18
19         public void addEdge_2017(String node1_2017, String node2_2017) {
20             graph_2017.putIfAbsent(node1_2017, new ArrayList<>());
21             graph_2017.putIfAbsent(node2_2017, new ArrayList<>());
22             graph_2017.get(node1_2017).add(node2_2017);
23             graph_2017.get(node2_2017).add(node1_2017);
24         }
25
26         public List<String[]> getAllEdges_2017() {
27             List<String[]> edges_2017 = new ArrayList<>();
28             Set<String> added_2017 = new HashSet<>();
29             for (String node_2017 : graph_2017.keySet()) {
30                 for (String neighbor_2017 : graph_2017.get(node_2017)) {
31                     String key1_2017 = node_2017 + "-" + neighbor_2017;
32                     String key2_2017 = neighbor_2017 + "-" + node_2017;
33                     if (!added_2017.contains(key1_2017) && !added_2017.contains(key2_2017)) {
34                         edges_2017.add(new String[]{node_2017, neighbor_2017});
35                         added_2017.add(key1_2017);
36                     }
37                 }
38             }
39             return edges_2017;
40         }
41     }
42 }

```

2. static class GraphTraversal_2017 menggunakan LinkedHashMap agar urutan penyisipan node tetap aktif, addEdge_2017 menambahkan sisi dua arah, getAllEdges_2017 metode helper untuk mengambil semua sisi yang unik.

```

// BFS iteratif
public TraversalResult_2017 bfs_2017(String start_2017, String goal_2017) {
    List<String> visited_2017 = new ArrayList<>();
    Map<String, String> parent_2017 = new LinkedHashMap<>();
    Queue<String> queue_2017 = new LinkedList<>();
    queue_2017.add(start_2017);
    visited_2017.add(start_2017);
    parent_2017.put(start_2017, null);
    while (!queue_2017.isEmpty()) {
        String current_2017 = queue_2017.poll();
        if (current_2017.equals(goal_2017)) break;
        for (String neighbor_2017 : graph_2017.getOrDefault(current_2017, new ArrayList<>())) {
            if (!visited_2017.contains(neighbor_2017)) {
                queue_2017.add(neighbor_2017);
                visited_2017.add(neighbor_2017);
                parent_2017.put(neighbor_2017, current_2017);
            }
        }
    }
    return new TraversalResult_2017(visited_2017, buildPath_2017(parent_2017, start_2017, goal_2017));
}

```

3. Algoritma BFS

```

// DFS rekursif
public TraversalResult_2017 dfs_2017(String start_2017, String goal_2017) {
    List<String> visited_2017 = new ArrayList<>();
    Map<String, String> parent_2017 = new LinkedHashMap<>();
    parent_2017.put(start_2017, null);
    dfsHelper_2017(start_2017, goal_2017, visited_2017, parent_2017);
    return new TraversalResult_2017(visited_2017, buildPath_2017(parent_2017, start_2017, goal_2017));
}

private boolean dfsHelper_2017(String current_2017, String goal_2017,
    List<String> visited_2017, Map<String, String> parent_2017) {
    if (visited_2017.contains(current_2017)) return false;
    visited_2017.add(current_2017);
    if (current_2017.equals(goal_2017)) return true;
    for (String neighbor_2017 : graph_2017.getOrDefault(current_2017, new ArrayList<>())) {
        if (!visited_2017.contains(neighbor_2017)) {
            parent_2017.put(neighbor_2017, current_2017);
            if (dfsHelper_2017(neighbor_2017, goal_2017, visited_2017, parent_2017)) return true;
        }
    }
    return false;
}

```

4. Algoritma DFS


```

private List<String> buildPath_2017(Map<String, String> parent_2017,
    String start_2017, String goal_2017) {
    List<String> path_2017 = new ArrayList<>();
    if (!parent_2017.containsKey(goal_2017)) return path_2017;
    String cur_2017 = goal_2017;
    while (cur_2017 != null) {
        path_2017.add(0, cur_2017);
        cur_2017 = parent_2017.get(cur_2017);
    }
    return path_2017;
}

```

5. buildPath_2017 metode ini bekerja mundur dari goal ke start menggunakan Map parent_2017 lalu menyusunnya menjadi list jalur yang berurutan dari awal ke akhir

```

static class TraversalResult_2017 {
    public final List<String> visited_2017;
    public final List<String> path_2017;
    TraversalResult_2017(List<String> v, List<String> p) {
        visited_2017 = v;
        path_2017 = p;
    }
}

```

6. TraversalResult_2017 digunakan untuk membungkus algoritma BFS dan DFS

```

class GraphPanel_2017 extends JPanel {

    private static final long serialVersionUID = 2L;
    private static final int R_2017 = 30;

    @Override
    protected void paintComponent(Graphics g_2017) {
        super.paintComponent(g_2017);
        Graphics2D g2_2017 = (Graphics2D) g_2017;
        g2_2017.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

        drawEdges_2017(g2_2017);
        drawNodes_2017(g2_2017);
        drawLegend_2017(g2_2017);
    }

    private void drawEdges_2017(Graphics2D g2_2017) {
        for (String[] edge_2017 : traversal_2017.getAllEdges_2017()) {
            int[] p1_2017 = nodePos_2017.get(edge_2017[0]);
            int[] p2_2017 = nodePos_2017.get(edge_2017[1]);
            if (p1_2017 == null || p2_2017 == null) continue;
            if (isEdgeOnPath_2017(edge_2017[0], edge_2017[1])) {
                g2_2017.setColor(Color.ORANGE);
                g2_2017.setStroke(new BasicStroke(3f));
            } else {
                g2_2017.setColor(Color.DARK_GRAY);
                g2_2017.setStroke(new BasicStroke(1.5f));
            }
            g2_2017.drawLine(p1_2017[0], p1_2017[1], p2_2017[0], p2_2017[1]);
        }
        g2_2017.setStroke(new BasicStroke(1f));
    }
}

```

7. drawEdges_2017 menggambar garis antar node. Jika garis tersebut merupakan bagian dari jalur akhir (path), warnanya diubah menjadi oranye dan lebih tebal

```
private boolean isEdgeOnPath_2017(String a_2017, String b_2017) {
    if (pathNodes_2017.size() < 2) return false;
    for (int i_2017 = 0; i_2017 < pathNodes_2017.size() - 1; i_2017++) {
        String u_2017 = pathNodes_2017.get(i_2017);
        String v_2017 = pathNodes_2017.get(i_2017 + 1);
        if ((u_2017.equals(a_2017) && v_2017.equals(b_2017)) ||
            (u_2017.equals(b_2017) && v_2017.equals(a_2017))) return true;
    }
    return false;
}

private void drawNodes_2017(Graphics2D g2_2017) {
    for (String node_2017 : nodePos_2017.keySet()) {
        int[] pos_2017 = nodePos_2017.get(node_2017);
        int x_2017 = pos_2017[0];
        int y_2017 = pos_2017[1];

        Color fill_2017;
        if (!pathNodes_2017.isEmpty() && node_2017.equals(pathNodes_2017.get(0))) {
            fill_2017 = Color.GREEN;
        } else if (!pathNodes_2017.isEmpty() &&
            node_2017.equals(pathNodes_2017.get(pathNodes_2017.size() - 1))) {
            fill_2017 = Color.RED;
        } else if (pathNodes_2017.contains(node_2017)) {
            fill_2017 = Color.YELLOW;
        } else if (visitedNodes_2017.contains(node_2017)) {
            fill_2017 = Color.CYAN;
        } else {
            fill_2017 = Color.LIGHT_GRAY;
        }

        g2_2017.setColor(fill_2017);
        g2_2017.fillOval(x_2017 - R_2017, y_2017 - R_2017, R_2017 * 2, R_2017 * 2);
        g2_2017.setColor(Color.BLACK);
        g2_2017.setStroke(new BasicStroke(1.5f));
        g2_2017.drawOval(x_2017 - R_2017, y_2017 - R_2017, R_2017 * 2, R_2017 * 2);
        g2_2017.setStroke(new BasicStroke(1f));

        g2_2017.setFont(new Font("SansSerif", Font.BOLD, 10));
        g2_2017.setColor(Color.BLACK);
        FontMetrics fm_2017 = g2_2017.getFontMetrics();
        String[] parts_2017 = node_2017.split(" ", 2);
        if (parts_2017.length == 2) {
            g2_2017.drawString(parts_2017[0],
                x_2017 - fm_2017.stringWidth(parts_2017[0]) / 2, y_2017 - 3);
            g2_2017.drawString(parts_2017[1],
                x_2017 - fm_2017.stringWidth(parts_2017[1]) / 2, y_2017 + 11);
        } else {
            g2_2017.drawString(node_2017,
                x_2017 - fm_2017.stringWidth(node_2017) / 2, y_2017 + 4);
        }
    }
}
```

8. drawNodes_2017 memberikan warna yang berbeda pada node berdasarkan statusnya

```

private void drawLegend_2017(Graphics2D g2_2017) {
    int lx_2017 = 16;
    int ly_2017 = getHeight() - 105;
    g2_2017.setFont(new Font("SansSerif", Font.PLAIN, 11));
    String[][] items_2017 = {
        {"Start Node", "GREEN"},
        {"Goal Node", "RED"},
        {"Node pada Jalur", "YELLOW"},
        {"Dikunjungi", "CYAN"},
        {"Belum dikunjungi", "LIGHT_GRAY"}
    };
    Color[] colors_2017 = {Color.GREEN, Color.RED, Color.YELLOW, Color.CYAN, Color.LIGHT_GRAY};
    for (int i_2017 = 0; i_2017 < items_2017.length; i_2017++) {
        g2_2017.setColor(colors_2017[i_2017]);
        g2_2017.fillRect(lx_2017, ly_2017, 14, 14);
        g2_2017.setColor(Color.BLACK);
        g2_2017.drawRect(lx_2017, ly_2017, 14, 14);
        g2_2017.drawString(items_2017[i_2017][0], lx_2017 + 20, ly_2017 + 12);
        ly_2017 += 20;
    }
}

```

9. drawLegend_2017 menggambar legenda

```

20
21 private JComboBox<String> cbStart_2017;
22 private JComboBox<String> cbGoal_2017;
23 private JTextArea taResult_2017;
24 private GraphPanel_2017 graphPanel_2017;
25
26 private final GraphTraversal_2017 traversal_2017 = new GraphTraversal_2017();
27 private List<String> visitedNodes_2017 = new ArrayList<>();
28 private List<String> pathNodes_2017 = new ArrayList<>();
29 private final Map<String, int[]> nodePos_2017 = new LinkedHashMap<>();
30
31 private static final String[] NODES_2017 = {
32     "Garegeh", "Pasa Ateh", "Pasa Bawah", "Jam Gadang", "Panorama",
33     "Ngarai Sianok", "Kbn Binatang", "Blkg Balok", "Lapang Kantin", "Rmh Bung Hatta"
34 };
35
36
37 public static void main(String[] args) {
38     EventQueue.invokeLater(() -> {
39         try {
40             new PetaTempatWisata_2511532017().setVisible(true);
41         } catch (Exception e) { e.printStackTrace(); }
42     });
43 }
44

```

10. nodePos_2017 menyimpan koordinat X dan Y untuk setiap nama tempat wisata agar posisinya tetap saat digambar, NODES_2017 menyimpan nama tempat wisata

```

public PetaTempatWisata_2511532017() {
    setTitle("Pencarian Jalur Wisata Bukittinggi - BFS & DFS");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 900, 680);
    buildGraph_2017();
    setupUI_2017();
}

private void buildGraph_2017() {
    nodePos_2017.put("Garegeh", new int[]{ 90, 210});
    nodePos_2017.put("Pasa Ateh", new int[]{ 240, 80});
    nodePos_2017.put("Pasa Bawah", new int[]{ 240, 210});
    nodePos_2017.put("Jam Gadang", new int[]{ 400, 80});
    nodePos_2017.put("Panorama", new int[]{ 560, 80});
    nodePos_2017.put("Ngarai Sianok", new int[]{ 720, 80});
    nodePos_2017.put("Kbn Binatang", new int[]{ 560, 250});
    nodePos_2017.put("Blkg Balok", new int[]{ 720, 360});
    nodePos_2017.put("Lapang Kantin", new int[]{ 400, 360});
    nodePos_2017.put("Rmh Bung Hatta", new int[]{ 240, 360});

    traversal_2017.addEdge_2017("Garegeh", "Pasa Ateh");
    traversal_2017.addEdge_2017("Garegeh", "Pasa Bawah");
    traversal_2017.addEdge_2017("Garegeh", "Rmh Bung Hatta");
    traversal_2017.addEdge_2017("Pasa Ateh", "Pasa Bawah");
    traversal_2017.addEdge_2017("Pasa Ateh", "Jam Gadang");
    traversal_2017.addEdge_2017("Pasa Bawah", "Jam Gadang");
    traversal_2017.addEdge_2017("Pasa Bawah", "Rmh Bung Hatta");
    traversal_2017.addEdge_2017("Jam Gadang", "Panorama");
    traversal_2017.addEdge_2017("Jam Gadang", "Lapang Kantin");
    traversal_2017.addEdge_2017("Panorama", "Ngarai Sianok");
    traversal_2017.addEdge_2017("Panorama", "Kbn Binatang");
    traversal_2017.addEdge_2017("Ngarai Sianok", "Kbn Binatang");
    traversal_2017.addEdge_2017("Kbn Binatang", "Blkg Balok");
    traversal_2017.addEdge_2017("Blkg Balok", "Lapang Kantin");
    traversal_2017.addEdge_2017("Lapang Kantin", "Rmh Bung Hatta");
    traversal_2017.addEdge_2017("Ngarai Sianok", "Blkg Balok");
}

```

11. buildGraph_2017 method untuk mengatur tata letak visual dari 10 tempat, merangkai graf dengan menambahkan sisi yang menghubungkan tempat-tempat tersebut berdasarkan kedekatan.

```

private void setupUI_2017() {
    JPanel contentPane_2017 = new JPanel(new BorderLayout(5, 5));
    contentPane_2017.setBorder(new EmptyBorder(8, 8, 8, 8));
    setContentPane(contentPane_2017);

    // Panel atas
    JPanel topPanel_2017 = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
    topPanel_2017.setBorder(new TitledBorder("Pencarian Jalur Menggunakan BFS dan DFS"));

    topPanel_2017.add(new JLabel("Lokasi Awal :"));
    cbStart_2017 = new JComboBox<>(NODES_2017);
    topPanel_2017.add(cbStart_2017);

    topPanel_2017.add(new JLabel("Lokasi Tujuan :"));
    cbGoal_2017 = new JComboBox<>(NODES_2017);
    cbGoal_2017.setSelectedIndex(5);
    topPanel_2017.add(cbGoal_2017);

    JButton btnBFS_2017 = new JButton("BFS");
    JButton btnDFS_2017 = new JButton("DFS");
    JButton btnReset_2017 = new JButton("Reset");

    btnBFS_2017.addActionListener(e -> runBFS_2017());
    btnDFS_2017.addActionListener(e -> runDFS_2017());
    btnReset_2017.addActionListener(e -> resetGraph_2017());

    topPanel_2017.add(btnBFS_2017);
    topPanel_2017.add(btnDFS_2017);
    topPanel_2017.add(btnReset_2017);
    contentPane_2017.add(topPanel_2017, BorderLayout.NORTH);

    // Panel tengah
    GraphPanel_2017 = new GraphPanel_2017();
    graphPanel_2017.setBorder(new TitledBorder("Visualisasi Graph"));
    contentPane_2017.add(graphPanel_2017, BorderLayout.CENTER);

    // Panel bawah
    JPanel bottomPanel_2017 = new JPanel(new BorderLayout());
    bottomPanel_2017.setBorder(new TitledBorder("Hasil Pencarian"));
    bottomPanel_2017.setPreferredSize(new Dimension(0, 160));

    taResult_2017 = new JTextArea();
    taResult_2017.setEditable(false);
    taResult_2017.setFont(new Font("Monospaced", Font.PLAIN, 12));
    bottomPanel_2017.add(new JScrollPane(taResult_2017), BorderLayout.CENTER);
    contentPane_2017.add(bottomPanel_2017, BorderLayout.SOUTH);
}

```

12. setupUI_2017 mengatur tata tata letak jendela menggunakan BorderLayout

```

private void runBFS_2017() {
    String start_2017 = (String) cbStart_2017.getSelectedItem();
    String goal_2017 = (String) cbGoal_2017.getSelectedItem();
    if (start_2017.equals(goal_2017)) { showSameNodeWarning_2017(); return; }
    GraphTraversal_2017.TraversalResult_2017 res_2017 =
        traversal_2017.bfs_2017(start_2017, goal_2017);
    visitedNodes_2017 = res_2017.visited_2017;
    pathNodes_2017 = res_2017.path_2017;
    displayResult_2017("BFS", start_2017, goal_2017, res_2017.visited_2017, res_2017.path_2017);
    graphPanel_2017.repaint();
}

private void runDFS_2017() {
    String start_2017 = (String) cbStart_2017.getSelectedItem();
    String goal_2017 = (String) cbGoal_2017.getSelectedItem();
    if (start_2017.equals(goal_2017)) { showSameNodeWarning_2017(); return; }
    GraphTraversal_2017.TraversalResult_2017 res_2017 =
        traversal_2017.dfs_2017(start_2017, goal_2017);
    visitedNodes_2017 = res_2017.visited_2017;
    pathNodes_2017 = res_2017.path_2017;
    displayResult_2017("DFS", start_2017, goal_2017, res_2017.visited_2017, res_2017.path_2017);
    graphPanel_2017.repaint();
}

private void resetGraph_2017() {
    visitedNodes_2017.clear();
    pathNodes_2017.clear();
    taResult_2017.setText("");
    cbStart_2017.setSelectedIndex(0);
    cbGoal_2017.setSelectedIndex(5);
    graphPanel_2017.repaint();
}

```

```

private void resetGraph_2017() {
    visitedNodes_2017.clear();
    pathNodes_2017.clear();
    taResult_2017.setText("");
    cbStart_2017.setSelectedIndex(0);
    cbGoal_2017.setSelectedIndex(5);
    graphPanel_2017.repaint();
}

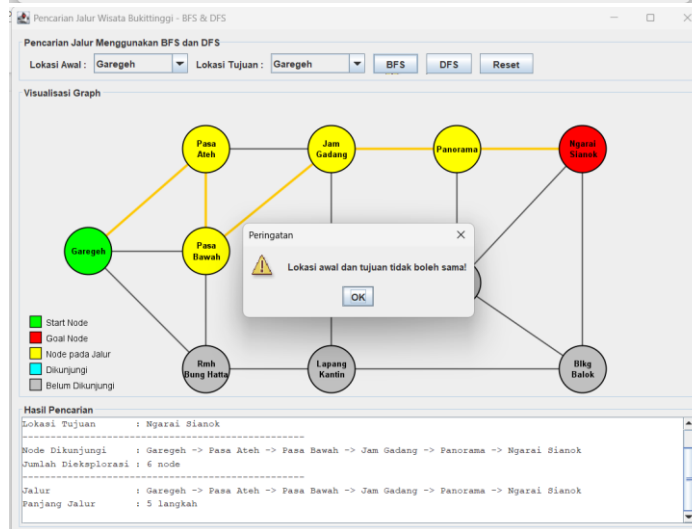
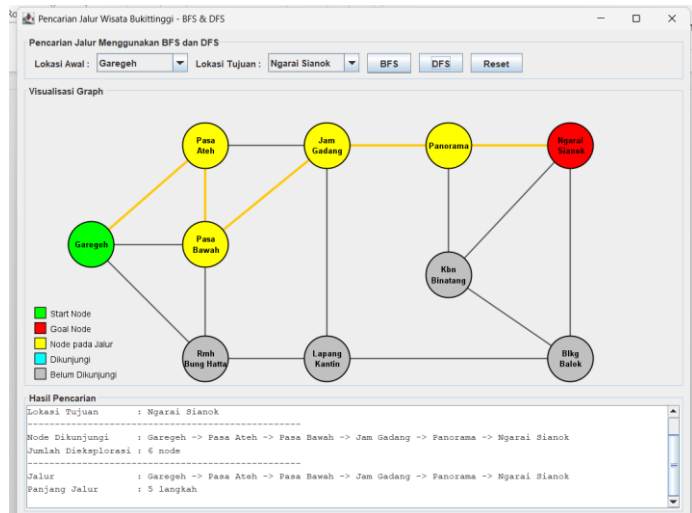
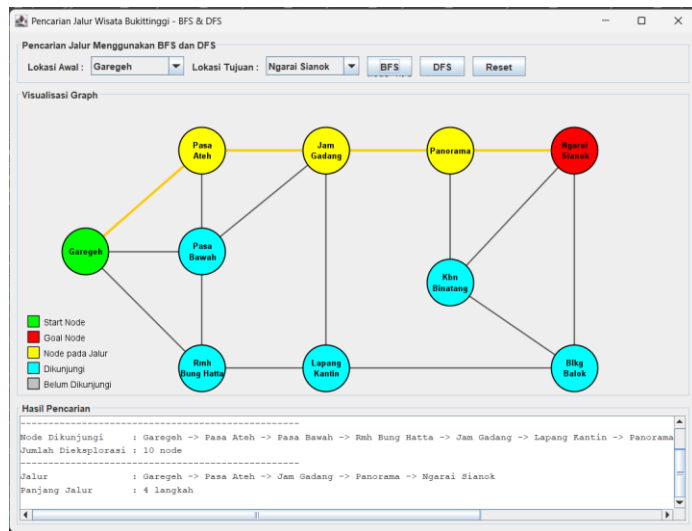
private void displayResult_2017(String algo_2017, String start_2017, String goal_2017,
    List<String> visited_2017, List<String> path_2017) {
    StringBuilder sb_2017 = new StringBuilder();
    sb_2017.append("Algoritma : ").append(algo_2017).append("\n");
    sb_2017.append("Lokasi Awal : ").append(start_2017).append("\n");
    sb_2017.append("Lokasi Tujuan : ").append(goal_2017).append("\n");
    sb_2017.append("-----\n");
    sb_2017.append("Node Dikunjungi : ").append(String.join(" -> ", visited_2017)).append("\n");
    sb_2017.append("Jumlah Dieksplorasi : ").append(visited_2017.size()).append(" node\n");
    sb_2017.append("-----\n");
    if (path_2017.isEmpty()) {
        sb_2017.append("Jalur : Tidak ditemukan!\n");
    } else {
        sb_2017.append("Jalur : ").append(String.join(" -> ", path_2017)).append("\n");
        sb_2017.append("Panjang Jalur : ").append(path_2017.size() - 1).append(" langkah\n");
    }
    taResult_2017.setText(sb_2017.toString());
}

private void showSameNodeWarning_2017() {
    JOptionPane.showMessageDialog(this,
        "Lokasi awal dan tujuan tidak boleh sama!",
        "Peringatan", JOptionPane.WARNING_MESSAGE);
}
}

```

13. runBFS_2017 dan runDFS_2017 bekerja dengan cara yang sama mengambil input start dan goal, validasi dan menjalankan algoritma dan menyimpan hasilnya. graphPanel_2017.repaint() untuk visualisasi

14. Output



BAB III

KESIMPULAN

Berdasarkan hasil dari praktikum dapat disimpulkan bahwa implementasi struktur data non linear yaitu Binary Tree dan Graph telah berhasil dilakukan dengan baik. Pada bagian *Binary Tree*, praktikan berhasil membangun struktur data hierarkis melalui pembuatan kelas *Node* dan penghubungan antar-simpul (*linking*) untuk membentuk pohon. Pemahaman mendalam diperoleh melalui implementasi tiga metode penelusuran secara rekursif, yaitu Pre-order, In-order, dan Post-order. Hasil pengujian menunjukkan bahwa urutan pemrosesan data sangat bergantung pada algoritma penelusuran yang dipilih, yang masing-masing memiliki kasus penggunaan spesifik dalam pengelolaan data dinamis. Pada Graph kita berhasil mengaplikasikan penelusuran BFS dan DFS untuk mencari rute terbaik.

DAFTAR PUSTAKA

- [1] M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: John Wiley & Sons, 2014.
- [2] R. Munir, *Struktur Data*. Bandung, Indonesia: Penerbit Informatika, 2019.